

AI 활용을 위한 게임 개발 구조화

AI 활용을 위해 먼저 개발 구조를 설계한다

생성 속도를 높이는 것보다, 무엇을 생성해도 흔들리지 않는 경계를 먼저 만든다.

핵심 전환

AI를 한 명의 개발자로 보지 않는다.

명확한 경계와 검증 가능한 인수인계를 따라 일하는 역할 집합으로 사용한다.

01 모호한 요청

Planning Agent | 목표·범위·완료 조건을 카드로 고정한다.

02 큰 작업과 충돌

역할·격리 | 코드·아트·실행·테스트의 변경 경계를 나눈다.

03 컨텍스트 단절

AnchorMind | 결정·오류·절차를 다음 역할에 넘긴다.

04 완료를 믿기 어려움

증거·QA·승인 | 자동 검증과 사람의 최종 판단을 연결한다.

구조가 고정하는 순서

의도

범위

역할

증거

승인

기억

결과

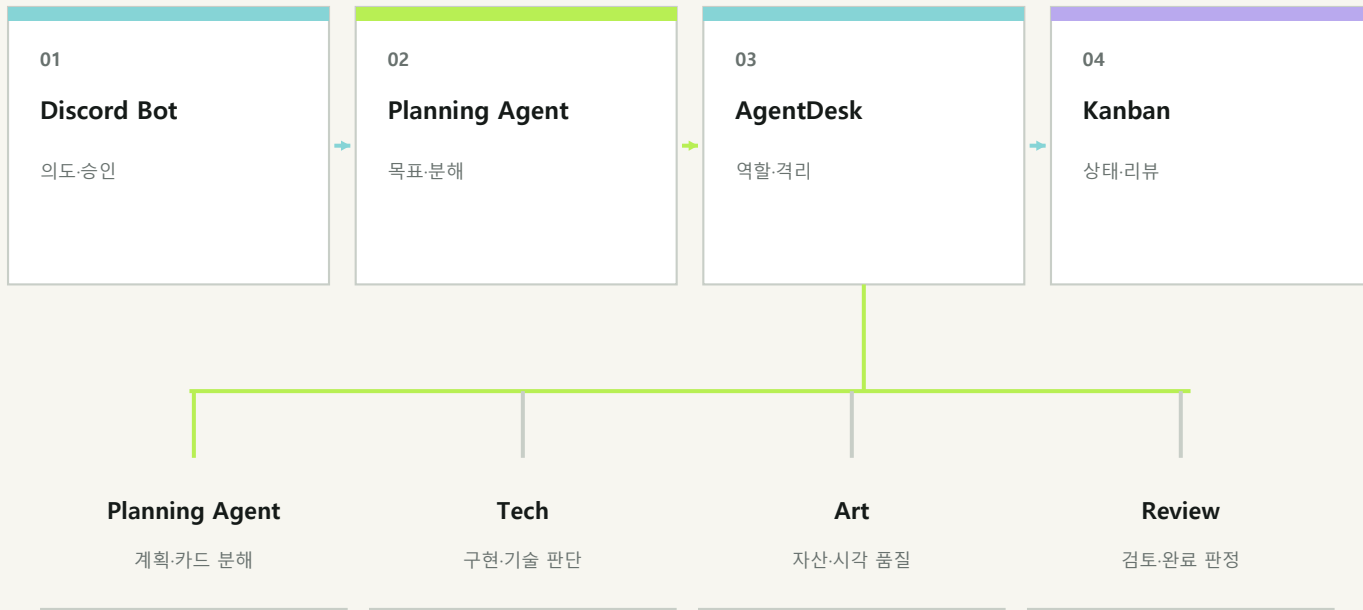
빠르게 만드는 구조가 아니라,
AI를 안전하게 활용할 수 있는 구조를 만든다.

자동화는 속도를 높이고, 구조는 방향을 지킨다.

사람의 의도가 작업 단위와 증거로 변환되는 경로

각 구성요소는 한 가지 책임만 맡고, 결과는 다음 단계가 확인할 수 있는 형태로 남긴다.

REQUEST -> EXECUTION -> EVIDENCE



개발 중 보이는 결과

카드

목표·완료 조건·상태

작업 공간

담당 역할·변경 경계

검증 기록

compile-test·실제 플레이

기억

결정·오류·절차

핵심 원칙

AI의 결과는 다음 역할이 읽고, 확인하고, 이어서 쓸 수 있어야 한다.

역할을 나누는 이유는 책임을 선명하게 만들기 위해서다

한 에이전트가 모든 것을 바꾸는 대신, 각 역할이 담당 범위와 검증 결과를 분리해 남긴다.

ONE TASK / FOUR RESPONSIBILITIES

01 Planning Agent

목표를 카드로 바꾸고 우선순위·의존성·완료 조건을 정한다.

02 Tech Agent

격리된 공간에서 구현하고 변경·검증 결과를 남긴다.

03 Art Agent

자산을 만들고 적용 경로와 시각 품질 기준을 확인한다.

04 Reviewer / QA

구현자와 분리된 관점에서 증거와 완료 조건을 검토한다.

칸반은 상태 머신이다



REVIEW -> DOING / 보완이 필요한 경우

역할 분리가 만드는 효과

작업 단위

AI가 처리할 범위와 완료 조건을 먼저 고정한다.

변경 경계

역할별 공간에서 충돌과 책임을 분리한다.

검증 연결

코드·테스트·플레이 결과를 같은 카드에 남긴다.

대화를 추적 가능한 작업으로 바꾼다

사람은 자연어로 시작하고, 시스템은 목표·담당·상태·증거가 남는 개발 흐름으로 변환한다.

A REQUEST BECOMES A TRACE

대화에서 작업으로

01 USER

새 기능 개발을 시작해줘.

02 DISCORD BOT

목표와 완료 조건을 확인했습니다.
Planning 카드로 분해합니다.

03 DISCORD BOT

Planning 결과를 Tech·Art Agent에 배정했습니다.
각 작업은 격리된 공간에서 진행합니다.

04 USER

확인했어. 승인할게.

05 DISCORD BOT

검증을 통과했습니다.
변경과 증거를 기록하고 기억에 공유합니다.

변환되는 책임

사람의 문장

목표·완료 조건

Planning

카드·의존성

AgentDesk

담당·격리 공간

Review

테스트·증거

승인

사람의 최종 판단

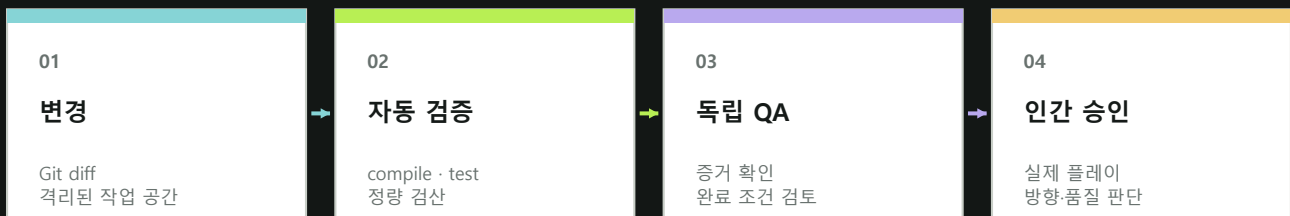
Discord Bot의 역할

요청을 접수하고, 상태를 알리고, 승인과 증거를 같은 흐름에 묶는다.

AI 결과는 증거와 인간 판단으로 닫힌다

반복 가능한 사실은 자동으로 확인하고, 게임의 감각과 방향은 사람이 최종 승인한다.

검증 루프



완료 선언이 아니라 변경·검증·판정의 흔적을 남기는 것이 종료 조건이다.

실제 작업에서 분리한 질문

변경이 의도한 범위인가?

카드·완료 조건·Git diff

Planning · Human

구현이 실제로 동작하는가?

compile · EditMode/PlayMode · raw evidence

Tech · QA

게임으로서 괜찮은가?

실제 플레이 · 시각 검토 · 방향 피드백

Human

실제 작업에 적용한 방식

UI

아트 자산·UGUI 연결·회귀 테스트·실제 플레이

씬·배치

기획 도안·기술 재빌드·정량 검사·시각 검토

네트워크

host-client 범위와 후속 Relay/Lobby를 분리

다음 역할이 같은 기준으로 이어서 일하도록 기억을 공유한다

현재 카드의 문맥과 팀의 장기 지식을 분리해 저장하고, 검증된 기준을 다음 작업으로 넘긴다.

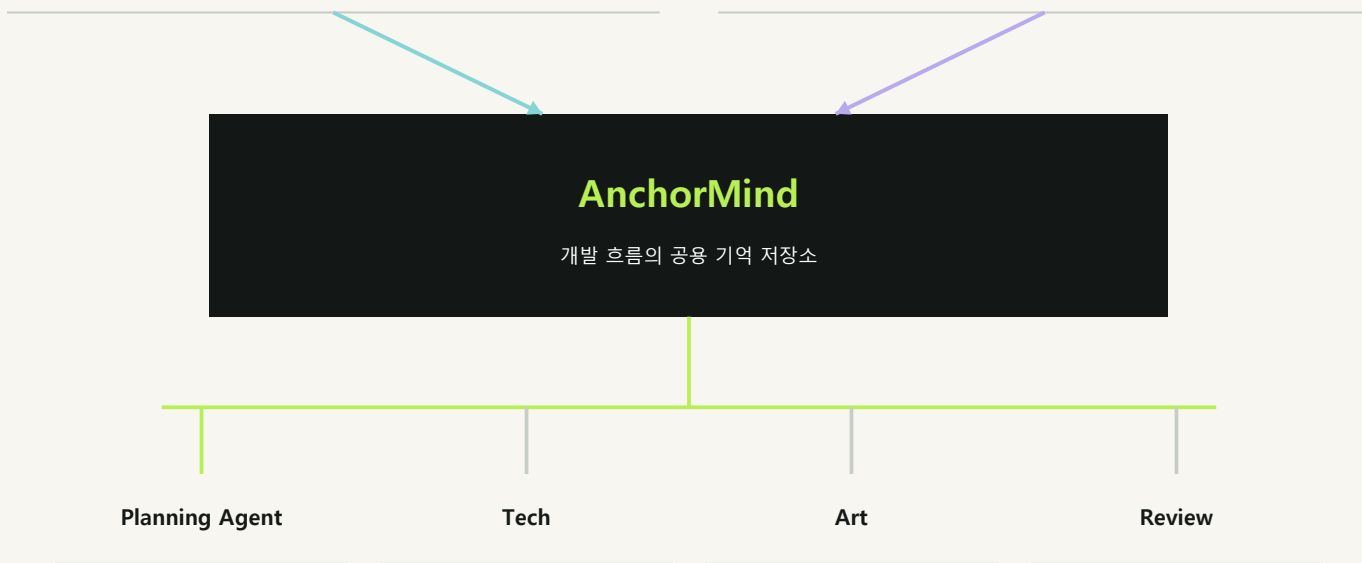
TWO TYPES OF MEMORY / ONE SHARED CONTEXT

작업 기억 | 단기

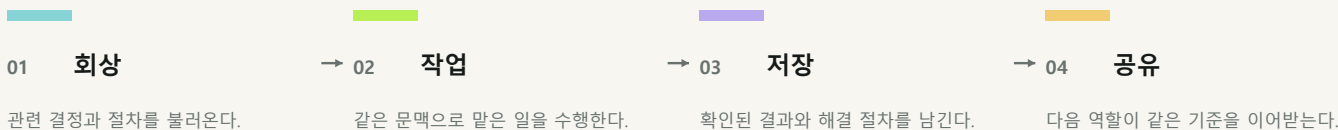
현재 카드의 목표, 진행 상태, 남은 문제, 다음 역할에 넘길 내용을 저장한다.

개발 기억 | 장기

확정된 결정, 팀 규칙, 해결한 오류, 반복해서 사용할 절차를 보존한다.



기억이 이어지는 순서



사람이 의도를 정하고, 역할이 구현하고, 증거와 기억이 다음 작업을 잇는다.